Taller Express y otras cosas

Nombre: Victor Manuel Laureano Vega

Matrícula: A01638979

Película favorita y razón: *Sans soleil*, de Chris Marker. Vi esta película (que es una especie de documental-ensayo) a principios del año pasado, y desde entonces se convirtió en mi favorita por las ideas que expone sobre los recuerdos y el tiempo.

El objetivo de la actividad es explorar y conocer más sobre el desarrollo web del lado del backend.

Al final de la actividad deberás hacer un PDF con este documento. El PDF deberá tener un enlace en supertarea.html

- **Ejercicio 0: El pasado**

En las semanas previas vimos la forma de consumir un API desde el cliente (el navegador). Ahora consumirás el **API desde el servidor**. Usa javascript y nodejs.

Selecciona un API que no requiera autenticación.

El servidor deberá consumir el API, procesarlo de alguna manera (ejemplo: seleccionar un dato) y posteriormente entregar ese dato (o datos) al cliente (navegador).

Sube tu archivo a tu repositorio.

Escribe aquí abajo el enlace al archivo en tu repositorio:
https://github.com/victorlaureanovega/victorlaureanovega.github.io/blob/main/Clase_25-03-2025/server.js

¿Cuál fue el principal problema para resolver este ejercicio? ¿Cuál fue el problema que tuvo una de las personas en tu equipo? Explica cómo resolvió esa persona ese problema.
No tuve ninguna dificultad especial al momento de realizar este código. Me basé en los que hemos escrito en clases previas.

- **Ejercicio 1: Simplicidad**

¿Recuerdas cómo **instalar un módulo** en node.js? Escribe la instrucción que se usa:
npm install [nombre del paquete]

- **Ejercicio 2: Express es**

Investiga qué es express (en el contexto de desarrollo web en nodejs). Explica a un **niñ@** qué es express, cómo se usa, para qué sirve. Escribe ese texto aquí. Incluye también **las referencias** de tu investigación.
Express es un marco de trabajo escrito en el lenguaje Javascript y almacenado en Node, que, antes de ser usado, debe descargarse usando el comando del punto anterior. Su propósito es brindar a los desarrolladores las herramientas mínimas necesarias para construir de forma sencilla servidores HTTP, con los que pueden hacerse aplicaciones web.

Referencias:
express (s.f.) Recuperado de express - npm

Express web framework (s.f.) Recuperado de Express web framework (Node.js/JavaScript) - Learn web development | MDN

Dile a una persona de la clase (que no esté en tu mesa) que te explique qué es express y qué ventajas te podría dar su uso. Graba el audio de la explicación.
https://drive.google.com/file/d/1sUKm_nqACoNDn4A2g1IKXoFVPaKyperN/view?usp=sharing

¿Crees que realizó una buena explicación? ¿Por qué?
Creo que es una explicación bastante general, aunque también me parece que abarca los puntos más importantes para formarnos una idea con la que podamos comenzar a investigar por nuestra cuenta, así que, en ese sentido, me parece buena.

- **Ejercicio 3: Desde un libro**

Investiga una definición en un libro (digital o físico) **de una API**. Incluye la referencia del libro.
Una API (Application Programming Interface) sirve para conectar procesos, servicios, contenidos y datos de una forma segura y relativamente sencilla. El libro proponía la comparación entre las APIs y barcos de carga que transportan contenido entre puertos.

Referencia:
Nijim, S., Pagano, B. (2014) *APIs for dummies*. Recuperado de https://cloud.google.com/files/apigee/apigee-apis-for-dummies-ebook.pdf

Escribe un ejemplo de un endpoint de tu proyecto.
/verificarusuarios

- **Ejercicio 4: Instala express**

Escribe la instrucción que se usa para **instalar el módulo de express:**
npm install express

- **Ejercicio 5: Uso de express**

Ejecuta y explica que hace el siguiente código:

```
import express from 'express';

const app = express();

app.listen(1984, () => {
   console.log('Up and up');
});
```

Este código sólo crea y habilita un servidor en el puerto 1984. Cuando esto sucede, se imprime en consola el mensaje “Up and up”.

- **Ejercicio 6: Uso de express++**

Ejecuta y explica que hace el siguiente código:

```
app.get('/bienvenida', (req, res) => {
   res.send('Esto no es una página html');
});

app.get('/otraBienvenida', (req, res) => {
  res.sendFile('bienvenida.html');
});
```

En caso de que alguno de estos te marque error, soluciona el problema y explica la forma de solucionarlo:

El segundo código me dio error, específicamente en las últimas tres líneas. Para solucionarlo, necesité crear una carpeta public que contiene el archivo bienvenida.html. También usé el siguiente código:

```
import path from 'path';
import { fileURLToPath } from 'url';

const __dirname = path.dirname(fileURLToPath(import.meta.url));

app.use(express.static(__dirname + '/public'));

app.get('/otraBienvenida', (req, res) => {
   res.sendFile(path.join(__dirname, 'public', 'bienvenida.html'));
});
```

Al parecer, se necesitaban los módulos path, para manejar rutas de archivos, y url, que sirve para convertir a rutas de archivo válidas. La línea siguiente extrae la dirección del archivo actual (es decir, codigo.js) y, a partir de esta, el directorio, que es guardado en la variable __dirname. Posteriormente, la función express.static() le dice al programa que necesitará servir (más adelante se explica esto) archivos localizados en una carpeta public almacenada en el mismo directorio, que los hace accesibles desde el servidor. Por último, se le da la instrucción al programa de que, si se agrega /otraBienvenida a la dirección localhost:1984, buscará en la carpeta public el archivo bienvenida.html, que es desplegado.

- **Ejercicio 7: Recuerdo de imagen**

Recuerdas que la imagen no se podía ver en https://github.com/sgiomatec/act2025. Un servidor entrega respuestas a solicitudes realizadas por un cliente. Cuando el cliente solicita una imagen (u otro tipo de archivo) el servidor tiene que saber responder.

Express nos propone crear un directorio para los archivos que queremos usar, por ejemplo imágenes o archivos de hojas de estilo.

Investiga qué es express.static. Escribe tu investigación.
La función express.static() sirve para servir (proporcionar, encontrar, regresar) archivos estáticos. Recibe como parámetro el nombre del directorio donde el desarrollador desea que Express haga la búsqueda. Esto sirve para no definir manualmente una ruta para el archivo

Referencia:
https://www.digitalocean.com/community/tutorials/nodejs-serving-static-files-in-express

- **Ejercicio 8: Película**

Pregunta a una persona de otra mesa su película favorita y sus razones.

¿Ya viste esa película? ¿Te gusta? ¿La verías si no la has visto? ¿Por qué?
La película favorita de Paulina es *Interestelar* (2014). Yo vi esta película hace casi dos años. Me parece que, aunque su trama es sencilla, también es interesante, y se nota que fue hecha bastante a consciencia, sobre todo en el aspecto técnico. Aunque el final no me parece tan emotivo como se lo parece a ella, creo que cierra la película de buena forma.

- **Ejercicio 9: Transformación**

Usa express para transformar todo el código de servidor.js, nombra el archivo ahora servidor_express.js

Escribe aquí abajo el enlace al archivo en tu repositorio:
https://github.com/victorlaureanovega/victorlaureanovega.github.io/blob/main/Clase_25-03-2025/servidor_express.js

- **Ejercicio 10: Verbos**

En el contexto de desarrollo web, explica los siguientes verbos GET, POST, PUT, DELETE
GET sirve para obtener información de un servidor. POST, para enviar información a un servidor. PUT, para crear o reemplazar elementos (https://developer.mozilla.org/es/docs/Web/HTTP/Reference/Methods/PUT). DELETE, para remover propiedades de un objeto tipo JSON (https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Operators/delete)

Explica los siguientes códigos, en el contexto de desarrollo web:

1xx
2xx

3xx
4xx
5xx

1xx indica que la solicitud del navegador ha sido recibida y está siendo procesada, aunque no hay una respuesta definitiva. 2xx, que la solicitud fue recibida y procesada. 3xx, que se debe hacer algo adicional para completar la solicitud. 4xx, que hubo un error en la solicitud, por ejemplo, que estaba mal formulada. 5xx, que hubo un problema en el servidor, como que este se encuentra sobrecargado.

Explica con un ejemplo (relacionado con tu proyecto/reto) cómo se atendería una solicitud de cada uno de esos verbos usando express. Incluye al menos una respuesta 404, 201 y 200.

Como el código 404 indica que se intentó acceder a algo que no existe en el servidor, se debe desplegar un mensaje que le indique claramente al usuario lo que sucedió. También podría agregarse un mensaje y un diseño sencillo para que no se despliegue el HTML "crudo". Un código como este podría desplegarse si se busca una escuela o un donante que no existe. En ese caso, basta con escribir esto de forma clara para que el usuario que realizó la búsqueda pueda intentar otra vez, o desistir, si así lo prefiriera. También se podría intentar regresar alguna escuela o donante cuyo nombre se asemeje más al que buscó, aunque esto será más complicado de implementar. En este caso también tendríamos que decirle que, aunque no se encontró exactamente lo que buscaba, los resultados que se muestran son los más parecidos.

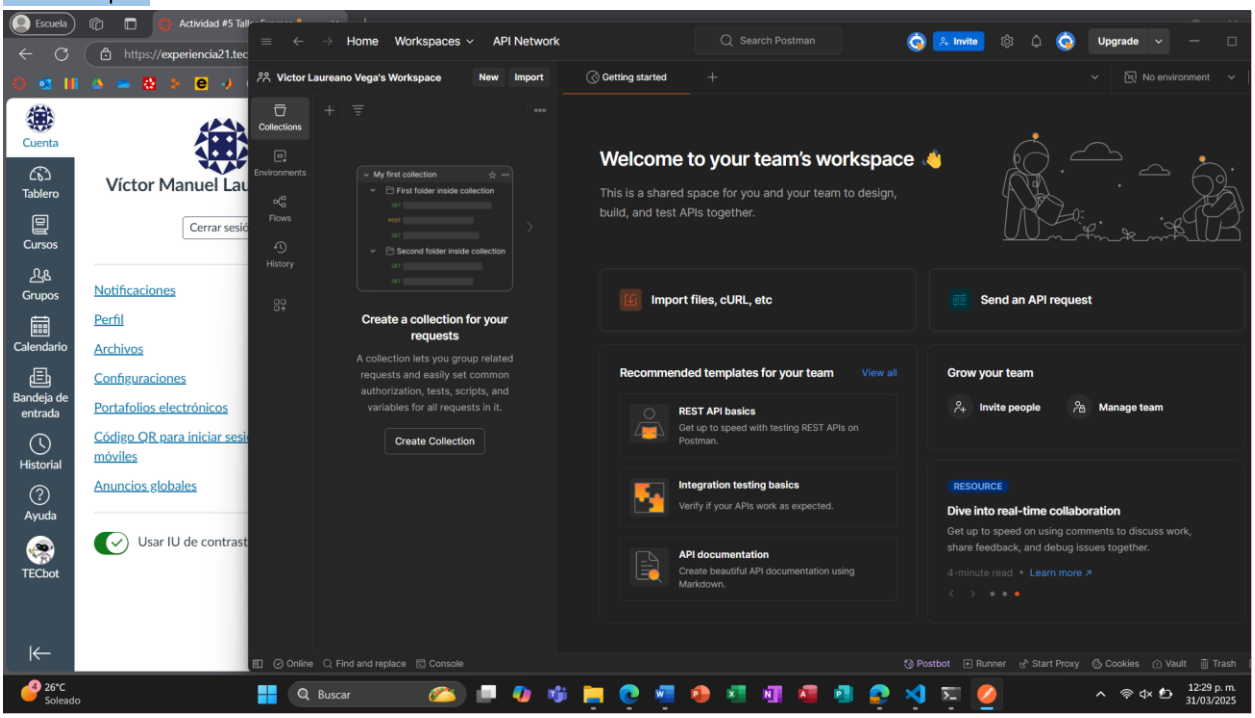
El código 201 podría aplicarse, por ejemplo, en la creación de un match entre escuela y donante. En este caso, a ambos se les desplegaría un mensaje que se los indique claramente; para uno, dirá el nombre de la escuela, y para otro, el del donante. También podría aplicarse en el envío de mensajes, pues este código indicaría que el mensaje ya se envió y ya existe la posibilidad de que el destinatario lo lea. Para ellos podríamos usar el lenguaje de palomitas usado en otras redes sociales, y que la mayoría de los usuarios ya conocerán.

Por su parte, el código 200 podría aplicar para aquellos en que la búsqueda encuentra un resultado exacto. En ese caso, debería regresársele al usuario indicándole que eso es lo que buscó. También podría aplicar para el inicio o cierre de sesión; en el primero, se cargaría su página adecuadamente, y en el segundo, regresaría al inicio de sesión habiendo guardado su actividad.

- **Ejercicio 11: Postman**

En equipo con tu mesa, instalen postman en cada una de sus computadoras. Tomen una misma foto en la que aparezcan todas las pantallas con postman abierto.

Foto aquí:



(Terminé esta actividad fuera de la clase)

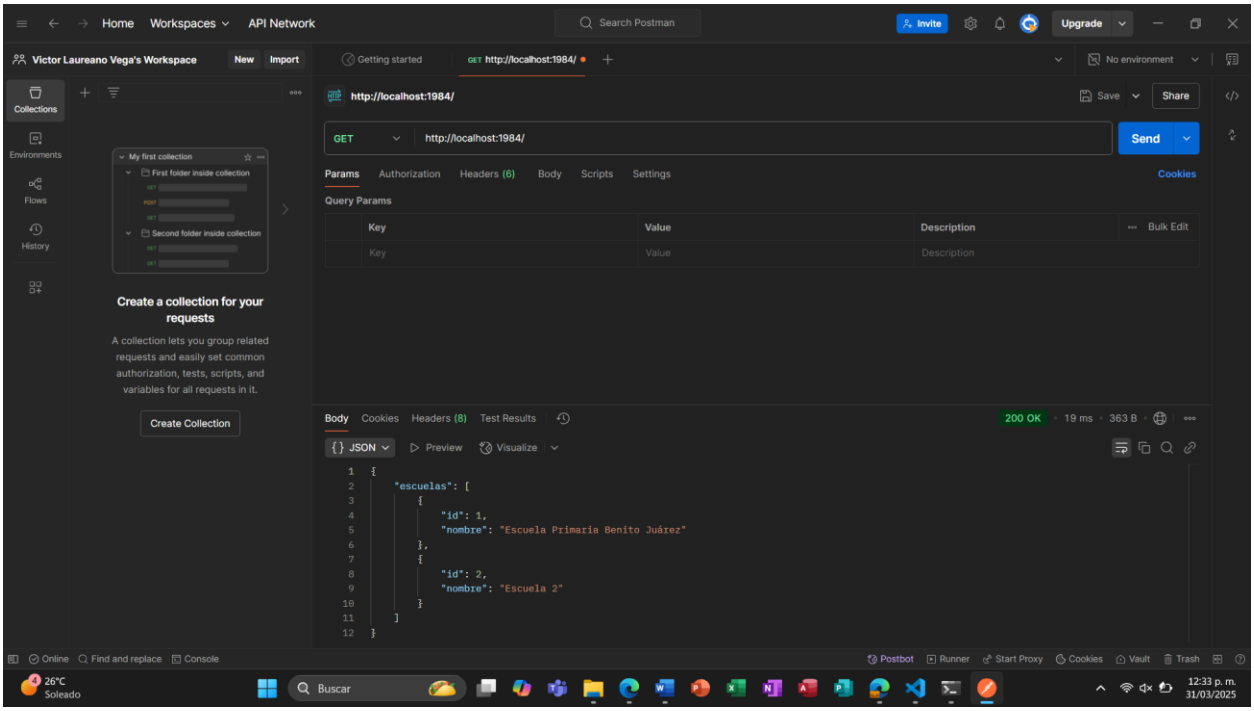
Explica para qué sirve Postman

Postman es una aplicación que sirve para para probar APIs haciendo peticiones de manera sencilla. También permite generar documentación para ellas y trabajar con casos de error, como los códigos 4xx o 5xx

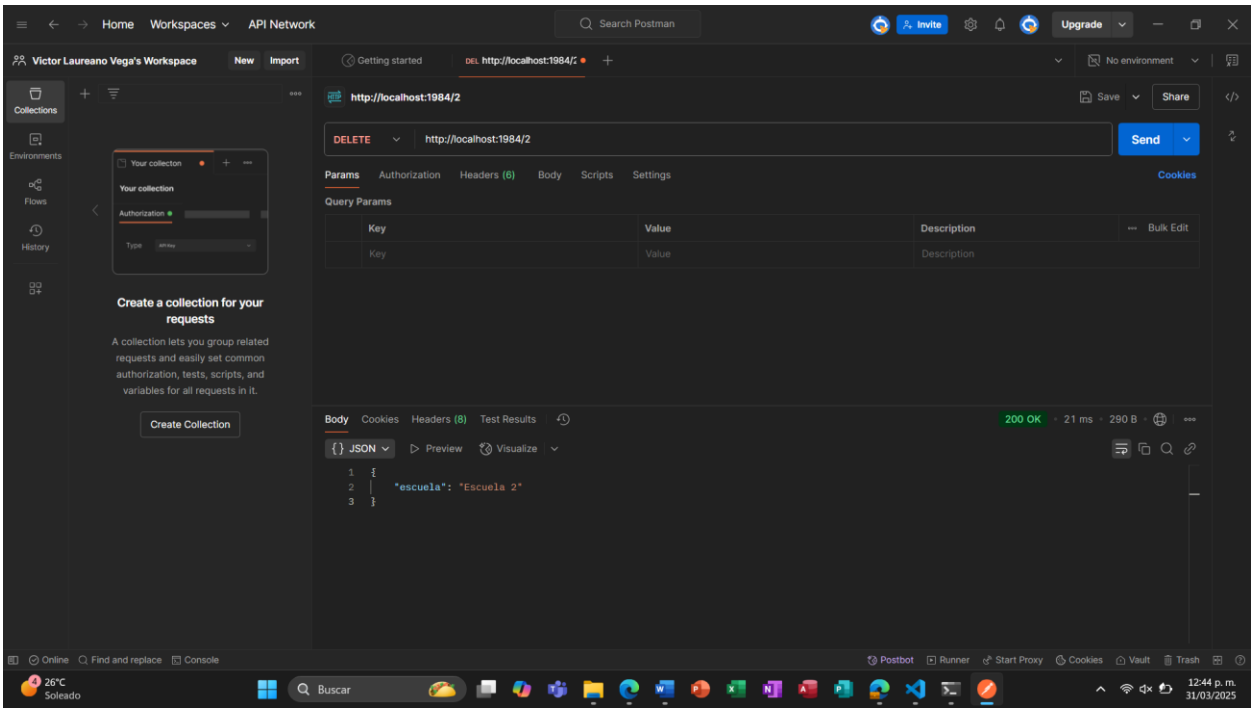
Prueba lo que hiciste en el ejercicio 10 con Postman. Pon una captura de pantalla usando el DELETE.

Probé lo que hice en el ejercicio 12 porque no entendí qué debía probar del 10.

Primero, un GET para demostrar que sí funciona:



A continuación, eliminé una de las escuelas:



- **Ejercicio 12: Parámetros**

Inventa un ejercicio (que esté relacionado con escuelas/donantes) para que uses parámetros en las rutas (por ejemplo /getEscuelas)

Soluciona ese ejercicio

Escribe aquí abajo el enlace al archivo en tu repositorio:
https://github.com/victorlaureanovega/victorlaureanovega.github.io/blob/main/Clase_25-03-2025/ejercicio12.js

Compara ese ejercicio con el de otra persona de la clase. ¿Qué ejercicio ayuda mejor a comprender el uso de parámetros y por qué?
Ambos ejercicios ayudaban a resolver un problema similar: el de encontrar escuelas y donadores. Sin embargo, únicamente por cuestiones de azar, implementé algunas herramientas que pueden ayudarnos con más cosas, como una herramienta que nos permite obtener la información de una sola escuela (que fácilmente puede convertirse en un filtro para donadores también) y otra que empareja escuelas y donadores.

- **Ejercicio 13: npx**

Investiga con alguien más lo siguiente: npx express-generator. Importante haz un directorio diferente si quieres ejecutar el comando.

Escribe el nombre de ese alguien más:
José Yael Varela García

Explica lo que genera y hace ese comando.
Este comando es una herramienta que permite generar la estructura básica de un proyecto de Express, algo así como un esqueleto del que podemos partir para no hacer el código desde cero.

Explica de manera básica y conceptualmente qué archivos se generaron.
Para hacer uso de este comando, necesitamos descargar el paquete express-generator. Tras crear un directorio nuevo, la ejecución de esta línea genera varios directorios nuevos, entre ellos: bin (el principal que funciona como punto de entrada y ejecución), public (que contiene los recursos estáticos que se usarán en el proyecto, como imágenes, hojas de estilos o códigos), routes (para las rutas que contienen una diagonal), views y los archivos app.js (con la configuración de Express) y package.json

- **Ejercicio 14: Explicar**

Explica a una niña o un niño de 10 años para qué sirven frameworks como Angular, React, Vue, o Svelte.
En general (y esto aplica, por supuesto, para proyectos de programación web), los frameworks sirven para desarrollar código con un poco de ventaja. Es decir, para no escribirlo desde cero. Los marcos de trabajo nos ofrecen algunas herramientas ya hechas (como funciones que podemos mandar llamar) que nos simplifican el trabajo. Esto implica, primero, que debemos conocer las reglas que establecieron los desarrolladores del framework, además de que debemos seguirlas pues de otra forma no podríamos hacer uso de su código.
Preferimos utilizarlos no sólo porque nos simplifican mucho el desarrollo de código (en cuestiones técnicas y de tiempo), sino porque, en ocasiones, también nos ayudan a detectar errores o a corregirlos de una forma más sencilla. Hacer todo desde cero requiere, primero, más habilidades técnicas, y, además, saber documentar adecuadamente el código, pues, ya que el desarrollo puede extenderse mucho tiempo, podríamos olvidar lo que hicimos o por qué lo hicimos.